VIRTUSLAB

The state of Scala in 2026

Where we are and where we are going



Piotr Chabelski
Product Manager @ Scala Core Team
https://github.com/Gedochao

Lukas Rytz
Compiler Engineer @ Scala Core Team
https://github.com/lrytz



About us

Piotr Chabelski
Scala Product Manager



Gedochao on GitHub

Scala CLI maintainer

Scala 3 compiler product manager

Scala Core team

Keeping the Scala 3 bug tracker in shape

Coordinating work across Scala OSS teams

Lukas Rytz
Scala Compiler Engineer



lrytz on GitHub

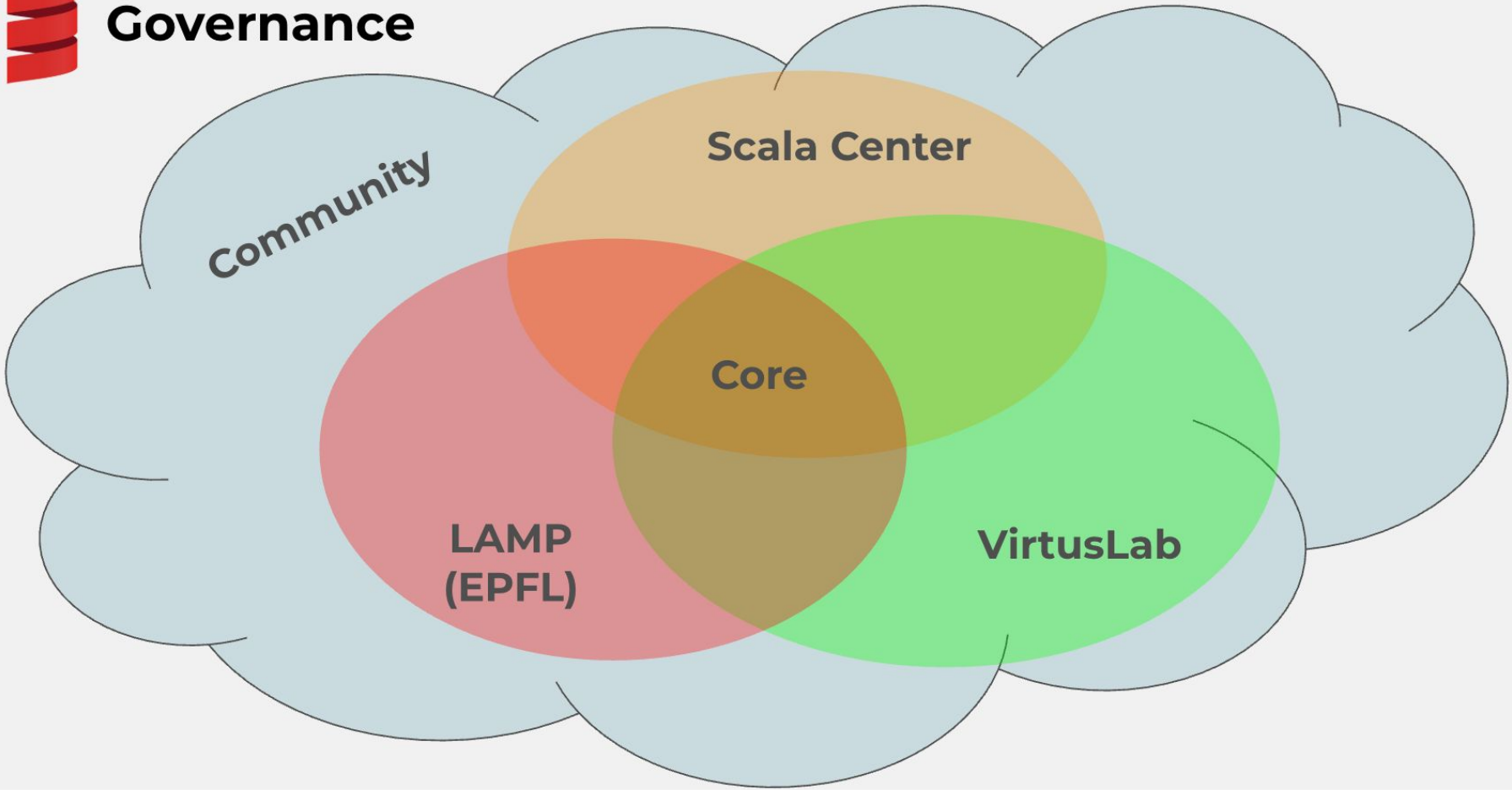
Working on Scala compilers since 2007

Scala 2 maintainer

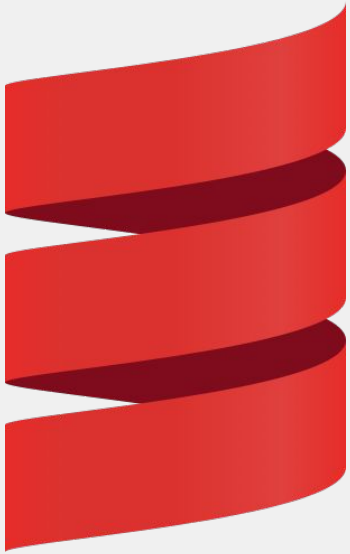
Scala Core team

SIP Committee member

Governance



Five Scala Distribution Lines



Supported and maintained

01 **Scala 2.12 (currently 2.12.21)**
In use by sbt 1.x

02 **Scala 2.13 (currently 2.13.18)**
Still widely used in production

03 **Scala 3.3 LTS (currently 3.3.8)**
The default ecosystem target until recently

04 **Scala 3.9 LTS (currently 3.9.0)**
The new LTS for the next 3+ years

05 **Scala Next (currently 3.10.0)**
The default version for up-to-date Scala apps

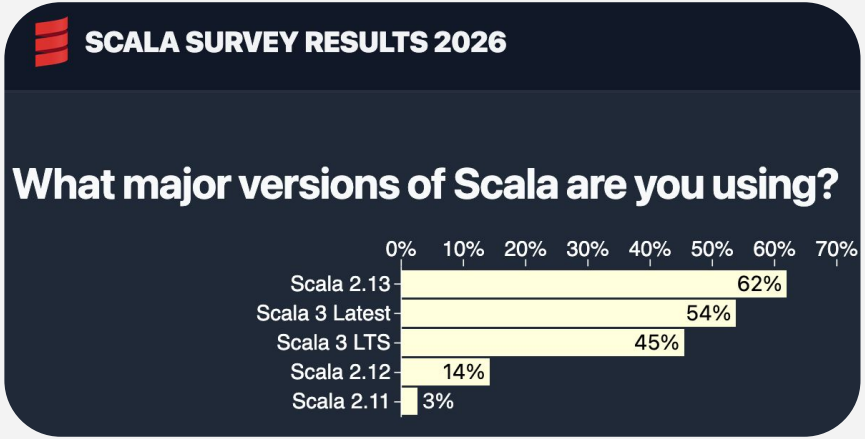


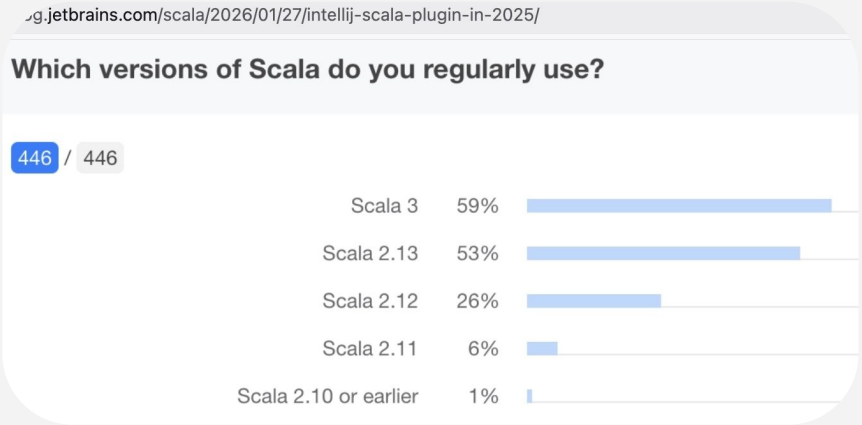
Scala 2.12

- Minimal maintenance, because **sbt 1.x** runs on Scala 2.12
- Using **sbt 1** is fine for now
- **sbt 2** was released in June 2026
  - The plugin ecosystem is catching up quickly
  - **sbt 2** requires Java 17, `-release` to target older JDKs
  - Scala 2.12 / 2.13 projects are supported
- Scala 2.12 in your project, **migrate to 2.13 now**:
  - Overhauled collections
  - `Seq` is now `immutable.Seq`

Scala 2.13

- Still widely used
  - Spark releases are on 2.13
    Usable with Scala 3
  - Chisel (hardware design)
    Scala 3 port in progress
- 2.13 Maintenance
  - JDK compatibility
  - Java interoperability
  - Security issues







Scala 2.13.19

- **2.13.19** is coming **after Scala Days**
  - (To be) tested on **JDK 27**
  - LazyList fix for memory usage on JDK 23+
  - Java Annotations on Java Types (JEP 104, Java 8)
  - (Possibly) Java value objects (JEP 401, preview in Java 28)
  - Library: fixes and improvements shared with Scala 3

- Contributions are welcome, reviewed, accepted!





Migrating from Scala 2.13 to 3.x

- Preparation on 2.13
  - JDK 17+, update **dependencies** (Scala 3 availability)
  - Replace: scala-reflect, Scala 2 macros (shapeless), plugins
  - Enable `-Xsource:3` (`-quickfix:cat=scala3-migration`)
- Migration tooling
  - **scala3-migrate** sbt plugin (Scala Center)
  - Binary interoperability: 2.13 reads 3.3 binaries, and vice versa
    - Limitations: macros, certain Scala 3 types





Migration Routes

- Migrate **modules one-by-one** to 3.3 (binary interoperability)
  - Classpath stays on Scala 2 deps (`for3Use2_13` in sbt)
- 3.3 ➡ 3.9 ➡ 3.10: **top-down** by module, or all at once
- Big bang: 2.13 ➡ 3.9 is realistic for small projects
- **Cross-build** 2.13 (with `-Xsource-features`) and 3.3 or 3.9
  - Pro: no interop issues (mixed classpath)
  - Con: two builds, overhead for entire team
- Offer by VirtusLab: 10 person-days of free migration support





Automatic rewrites

Compiler	Rewrite Flags
2.13.18	`-Xsource:3 -quickfix:cat=scala3-migration`
3.3.8	`-source:3.0-migration -rewrite`
3.7.4*	`3.4-, 3.6-, 3.7-migration`
3.9 / 3.10	`3.8- / 3.9- / 3.10-migration`

* some rewrites need 3.7, the last version with the 2.13 library





Scala 3.3 LTS (the previous LTS line)

- Introduced in May 2023
- Will be **EOL in September 2027**
- Currently at **3.3.8** (with **3.3.9** just around the corner)
- First Scala 3 LTS series
- Uses Scala 2.13's standard library
- JDKs 8 to 23
  - JDK 24+ with `-Yfuture-lazy-vals`





The trouble with JDK 24+ at Scala <3.8

- **JEP-471**: warnings about **`sun.misc.Unsafe`** on **JDK 24+**
- will likely error on future JDKs
  - still no error on **JDK 27**

```
> scala -S 3.3.8 --jvm 27 -e 'println("Hello")'
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by
scala.runtime.LazyVals$
WARNING: Please consider reporting this to the maintainers of class
scala.runtime.LazyVals$
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future
release
Hello
```






The trouble with JDK 24+ at Scala <3.8

- **recommendation**: Scala **3.3** projects should use **`-Yfuture-lazy-vals`**
  - necessary to include **`-java-output-version:x`**
  - output Java has to be **9+**, recommended **11** or **17**

```
> scala -S 3.3.8 --jvm 27 -e 'println("Hello")'
  \ -Yfuture-lazy-vals -java-output-version:17
Hello
```



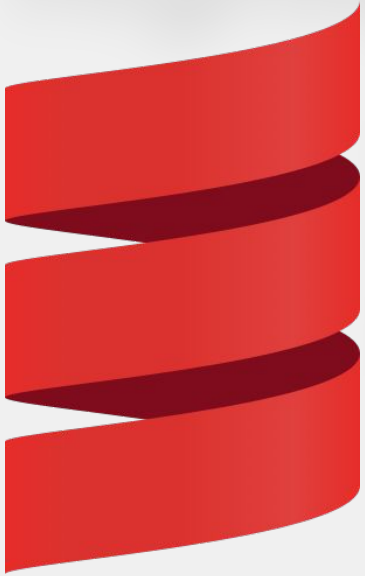



Scala 3.9 LTS (the current LTS line)

- Introduced in September 2026
- Currently at **3.9.0**
- Requires **JDK 17**
- **Recommended for libraries**
- New lazy vals implementation unlocks **JDK 24+**
- Standard library built with Scala 3
  - optimised with the newly ported optimizer
- Better tooling & IDE support
- Includes all the features introduced in Scala Next since 3.3



3.9 LTS Feature Highlights



just a few good reasons to migrate today

01 **Standard library built with Scala 3**
No more depending on Scala 2

02 **Bytecode optimizer**
Finally ported from Scala 2

03 **Better Fors (`SIP-62`)**
`for` comprehensions, but better

04 **Named Tuples (`SIP-58`)**
Practically, records with less boilerplate

05 **Scala CLI as `scala` (`SIP-46`)**
The new, shiny runner for the language



3.9 LTS Feature highlight: Standard library built with Scala 3

- Introduced in Scala 3.8
- Compiled with Scala 3
- Binary compatible with the old Scala 2.13 library
- Optimized with the bytecode optimizer in 3.9 LTS
- Fully capture checked (opt-in with capture checking enabled)
- With explicit null checks (opt-in with `-Yexplicit-nulls`)





3.9 LTS Feature highlight: Bytecode optimizer

- Available since Scala 3.8.3
- Ported from Scala 2
- Optimized standard library in **3.9.0**+
- Keep code high-level and let the optimizer then make it faster
- `-opt` / `-opt-inline:<pattern>`
  - note: only allow inlining for apps or deps you control

```
# Non-inlining optimisations only
scalac -opt -d out src/**/*.scala

# Allow inlining with package filters (example)
scalac -opt \
  -opt-inline:'**,!java.**,!javax.**' \
  -d out src/**/*.scala

# Optimizer help
scalac -opt-inline:help
```






3.9 LTS Feature highlight: Better Fors (SIP-62)

- Stable since Scala 3.8
- Improves readability of **`for`**-expressions
- The main syntactic sugar: value assignments now allowed at the start of a for comprehension

```
// without better fors
{
  val x = 1
  for {
    y <- Some(2)
  } yield x + y
}

// with better fors
for {
  x = 1 // alias allowed here
  y <- Some(2)
} yield x + y
```






3.9 LTS Feature highlight: Named Tuples (SIP-58)

- Stable since Scala 3.7
- easier access to tuple fields (`user.name` is nicer than `user._1`)
- names are a part of the type

```scala
val user = (name = "Gedochao", id = 1337)
println(user.name) // Gedochao
val text =
  user match
    case (name = n, id = i) =>
      s"Hello, user $n (id $i)"
println(text) // "Hello, Gedochao (id 1337)"
```






3.9 LTS Feature highlight: Scala CLI as the new `scala` runner (SIP-46)

- Stable since Scala 3.5
- Installed with the language
- Replaces the old **`scala`** runner
- **`compile`**, **`run`**, **`test`**, and **`package`** your Scala code (and more!)
- Easy build configuration with **`using directives`**
- We still claim it's not a build tool

```
scala -e 'println("Hello")'
scala . -S 3.8.3-RC3
scala compile .
scala package . --native --power
scala test .
scala publish . --power
scala repl --dep com.lihaoyi::os-lib:0.11.8
```



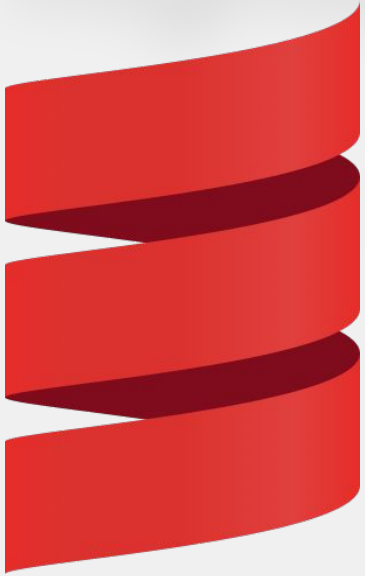



Scala Next

- Currently at **3.10.0**
- Includes all of the newest features, improvements and bug fixes
- Includes the latest `-experimental` and `-preview` features
- **Recommended for apps** (not libraries)
  - you get the fastest compiler available
  - you get the fewest bugs



Scala Next Feature Highlights



a small glimpse of the future

01 **Standard library open to contributions**
No longer just bug fixes

02 **Pattern matching for Java records**
`3.10`: Stable and ready to use.

03 **`@unroll` (`SIP-61`)**
`3.10` (`preview`): Extra arguments with binary compat

04 **Relaxed lambda syntax (`SIP-75`)**
`3.10` (`preview`): Single line lambdas with `:`

05 **Referenceable package objects (`SIP-68`)**
`3.10` (`preview`): Because why shouldn't they be

06 **Select `using` directives available in the REPL**
`3.10`: One can finally just `//> using dep`



Standard library is open to contributions!

- Has been since Scala **3.8** - but only for bug fixes
- All contributions are welcome starting with Scala **3.10**
- Contributions have to go through the **Standard Library Change** process (**SLC**)
  - create a contributors.scala-lang.org thread
  - create a Pull Request
  - wait for Scala Core approval





Scala Next Feature highlight: -> sugar made into an inline method

- available since Scala **3.10.0**
- old `Predef.ArrowAssoc` implicit class is deprecated
- we now have an inline extension method in `Predef.->`

```
// unchanged
val tuple1 = (5, 10)

// now produces the same bytecode as (5, 10)
val tuple2 = 5 -> 10
```






What does it mean that a feature is in `-preview`?

- `-preview` compiler setting to enable
- API and spec are already a part of the language
- Can be used in both apps and libraries
- They are no longer `-experimental`
- Gathering feedback before announcing as stable

```
scala compile . -preview
```






Scala Next Feature highlight (`-preview`): `@unroll` (SIP-61)

- `-preview` since Scala **3.10.0**
- a new annotation, which allows to:
  - add additional params to methods & constructors
  - avoid breaking binary compatibility

```
// Original
def foo(s: String, n: Int = 1, @unroll b: Boolean = true, @unroll l: Long = 0) =
 s + n + b + l

// Generated
def foo(s: String, n: Int, b: Boolean) = foo(s, n, b, 0)
def foo(s: String, n: Int) = foo(s, n, true, 0)
```






Scala Next Feature highlight (`-preview`): Relaxed lambda syntax (SIP-75)

- `-preview` since Scala **3.10.0**
- allows single-line lambdas after a `:`

```
// Previously
xs.map: x =>
  x + 1

// Single-line, with SIP-75
xs.map: x => x + 1
```






Scala Next Feature highlight (`-preview`): Referenceable package objects (SIP-68)

- `-preview` since Scala **3.10.0**
- allows for package objects to be referenced directly

```
package a
package object b

// workaround for pre-3.10
val foo = a.b.`package`

// with SIP-68
val foo = a.b
```






Scala Next Feature highlight: Select `using` directives available in the REPL

- available since Scala **3.10.0**
- some of Scala CLI's **`using`** directives are now available in the REPL
  - **`//> using dep`**
  - **`//> using toolkit`**
  - **`//> using options`**
  - **`//> using resource`**
  - **`//> using repository`**
  - **`//> using jar`**

```
scala> //> using toolkit default
       os.pwd
Resolved 2 dependencies (24 JARs)
val res0: os.Path = /Users/name
```

- multiple directives or **`:commands`** can be passed at once
- trailing code is also allowed





Further reading

- Scala development, distribution lines and guarantees: https://www.scala-lang.org/development/
- Blog post about 3.9 LTS being released: https://www.scala-lang.org/news/3.9/
- Blog post about porting the bytecode optimizer: https://www.scala-lang.org/blog/2026/03/23/porting-the-optimizer.html
- SLC (Standard Library Contributions process) https://www.scala-lang.org/blog/2026/02/05/standard-library-process.html
- JEP-471 (the lazy vals / JDK headache) - https://openjdk.org/jeps/471
- SIPs
  - Better Fors - https://docs.scala-lang.org/sips/better-fors.html
  - Scala CLI - https://docs.scala-lang.org/sips/scala-cli.html
  - Named Tuples - https://docs.scala-lang.org/sips/58.html
  - `@unroll` - https://docs.scala-lang.org/sips/unroll-default-arguments.html
  - Single line lambdas - https://docs.scala-lang.org/sips/75.html
  - Referenceable package objects - https://docs.scala-lang.org/sips/68.html





Thank You

Piotr Chabelski
Product Manager @ Scala Core Team
pchabelski@virtuslab.com

https://github.com/Gedochao

Lukas Rytz
Compiler Engineer @ Scala Core Team
lrytz@virtuslab.com

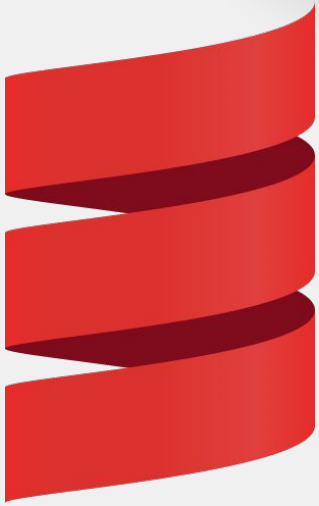
https://github.com/lrytz

Get the slides here!

slides QR

Q&A

with a list of stuff we touched on



01 **Five Scala distribution lines**
Those are **2.12**, **2.13**, **3.3 LTS**, **3.9 LTS** and **Scala Next**

02 **Governance**
Who even makes Scala these days?

03 **Migration from older Scala versions**
How do I get from **2.12** to **3.10**?

04 **Scala 2 support**
For how much longer can I stay at **2.12**?

05 **Scala 3.3 LTS going EOL in September 2027**
Less than a year of back porting bug fixes left.

06 **Going JDK 17 and beyond**
Do I really have to abandon Java 8?

07 **Scala 3.9 LTS**
Any questions on the new Long Term Support line?

08 **Scala Next / 3.10**
Or about the latest changes to the language?

09 **`-preview` features**
Can I use those in my project?